AI-Powered Smart Grid Digital Twin

Project source files and setup guide

Includes ESP32 starter firmware, Python AI training script, illustrative CSV dataset, Streamlit demo dashboard, requirements, README, and setup instructions.

Important: All sensor conversion factors require calibration. The sample data and dashboard readings are illustrative. Use only isolated low-voltage demo circuits during development; mains wiring must be handled by a qualified electrician.

README.md

```
# AI-Powered Smart Grid Digital Twin

A student prototype for monitoring low-voltage demonstration circuits using an ESP32, sensors, a
Python AI model, and a Streamlit dashboard.

## Features
- Read voltage, current, and transformer temperature (after sensor calibration).
- Demonstrate Normal, Overload, and Fault states.
- Display local readings on an OLED and a browser dashboard.
- Train a baseline Random Forest classifier from labelled data.
- Extend the system to receive live ESP32 data over Wi-Fi.

## Hardware
- ESP32 DevKit
- ZMPT101B voltage sensor module
- ACS712 current sensor module
- DS18B20 temperature sensor
- I2C OLED (often address 0x3C)
- Two-channel relay module
- Three push buttons
- Isolated low-voltage demo supply and loads

## Repository layout
- `esp32-code/SmartGrid.ino`: Arduino starter firmware
- `ai-model/train_model.py`: train/evaluate/save a Random Forest model
- `ai-model/smart_grid_data.csv`: small illustrative example dataset
- `dashboard/app.py`: Streamlit demonstration dashboard
- `requirements.txt`: Python dependencies

## Quick start: dashboard
1. Install Python 3.10 or later.
2. From this repository folder, run `pip install -r requirements.txt`.
3. Run `streamlit run dashboard/app.py`.
4. Use the sidebar to simulate conditions. These values are simulated, not live hardware readings.

## Train the AI model
From the repository folder:
`python ai-model/train_model.py`
This creates `ai-model/smart_grid_model.joblib`. The included CSV is tiny and illustrative only; it
is not suitable for real protection or credible accuracy claims. Collect a larger, correctly
labelled dataset and evaluate it on separate test data.

## ESP32 firmware
Open `esp32-code/SmartGrid.ino` in Arduino IDE. Install Adafruit SSD1306, Adafruit GFX Library,
OneWire, and DallasTemperature. Confirm your actual board/module pinouts before powering. Voltage
and current conversion factors are placeholders and must be calibrated.

## Safety
- The 230 V AC primary is hazardous. Mains wiring must be enclosed and handled by a qualified
electrician.
- Never connect mains voltage or an unconditioned sensor output directly to ESP32 GPIO.
- ESP32 analog inputs must stay within the board's permitted range (typically 0–3.3 V).
- ZMPT101B modules vary; confirm supply and input/output specifications.
- ACS712 boards commonly use 5 V; scale its output and verify the full waveform stays within the
ESP32 ADC range.
- Use only isolated, low-voltage demo loads during development. Do not rely on this prototype or
its AI output for real electrical protection.
```

requirements.txt

```
streamlit>=1.35
pandas>=2.0
scikit-learn>=1.3
joblib>=1.3
```

esp32-code/SmartGrid.ino

```
/*
  AI Smart Grid Digital Twin - ESP32 starter firmware
  Libraries: Adafruit SSD1306, Adafruit GFX, OneWire, DallasTemperature

  SAFETY:
  - Use only isolated low-voltage demo circuits while testing.
  - Do not connect mains voltage to ESP32 or a breadboard.
  - Verify analog sensor outputs stay within the ESP32 ADC input range.
  - Voltage/current conversion factors below are placeholders.
  - Relay polarity depends on the relay module; test with no load first.
*/

#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
```

```
#include <OneWire.h>
#include <DallasTemperature.h>
#include <math.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

const int TEMP_PIN = 4;
const int VOLTAGE_PIN = 35;  // ADC input only
const int CURRENT_PIN = 34;  // ADC input only
const int RELAY1_PIN = 25;
const int RELAY2_PIN = 26;
const int NORMAL_BTN = 27;
const int OVERLOAD_BTN = 14;
const int FAULT_BTN = 13;

OneWire oneWire(TEMP_PIN);
DallasTemperature tempSensor(&oneWire);

// PLACEHOLDERS: calibrate using appropriate isolated reference equipment.
float voltageFactor = 1.0;
float currentAmpsPerVolt = 1.0;
float currentZeroVolts = 1.65; // only valid if verified for your signal circuit

const float CURRENT_LIMIT_A = 2.0; // demonstration threshold only
const float TEMP_LIMIT_C = 60.0;   // demonstration threshold only

// Many relay modules are active LOW. Confirm your module before use.
const int RELAY_ON = LOW;
const int RELAY_OFF = HIGH;

float readVoltageRmsApprox() {
  const int samples = 1000;
  double sum = 0.0, sumSq = 0.0;
  for (int i = 0; i < samples; i++) {
    int raw = analogRead(VOLTAGE_PIN);
    sum += raw;
    sumSq += (double)raw * raw;
    delayMicroseconds(200);

  }
  double mean = sum / samples;
  double variance = sumSq / samples - mean * mean;
  if (variance < 0) variance = 0;
  float adcRms = sqrt(variance);
  float adcVoltsRms = adcRms * 3.3 / 4095.0;
  return adcVoltsRms * voltageFactor;
}

float readCurrentApprox() {
  const int samples = 500;
  long total = 0;
  for (int i = 0; i < samples; i++) {
    total += analogRead(CURRENT_PIN);
    delayMicroseconds(200);
  }
  float rawMean = total / (float)samples;
  float outputVolts = rawMean * 3.3 / 4095.0;
  return fabs(outputVolts - currentZeroVolts) * currentAmpsPerVolt;
}

void setDemoRelays(bool normalState) {
  // Demonstration only. Real protection requires independent, engineered protection.
  digitalWrite(RELAY1_PIN, normalState ? RELAY_ON : RELAY_OFF);
  digitalWrite(RELAY2_PIN, normalState ? RELAY_ON : RELAY_OFF);
}

void setup() {
  Serial.begin(115200);

  pinMode(NORMAL_BTN, INPUT_PULLUP);
  pinMode(OVERLOAD_BTN, INPUT_PULLUP);
  pinMode(FAULT_BTN, INPUT_PULLUP);

  digitalWrite(RELAY1_PIN, RELAY_OFF);
  digitalWrite(RELAY2_PIN, RELAY_OFF);
  pinMode(RELAY1_PIN, OUTPUT);
  pinMode(RELAY2_PIN, OUTPUT);

  analogReadResolution(12);
  analogSetPinAttenuation(VOLTAGE_PIN, ADC_11db);
  analogSetPinAttenuation(CURRENT_PIN, ADC_11db);

  Wire.begin(21, 22);
  tempSensor.begin();

  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("OLED not found. Check address and wiring.");
    while (true) delay(1000);
  }

  display.clearDisplay();
```

```
  display.setTextColor(SSD1306_WHITE);
  display.setTextSize(1);
  display.setCursor(0, 0);

  display.println("AI Smart Grid");
  display.println("Demo firmware");
  display.display();
  delay(1200);
}

void loop() {
  tempSensor.requestTemperatures();
  float temperature = tempSensor.getTempCByIndex(0);
  float voltage = readVoltageRmsApprox();
  float current = readCurrentApprox();

  bool normalPressed = digitalRead(NORMAL_BTN) == LOW;
  bool overloadPressed = digitalRead(OVERLOAD_BTN) == LOW;
  bool faultPressed = digitalRead(FAULT_BTN) == LOW;

  String status = "NORMAL";
  if (faultPressed) {
    status = "FAULT";
  } else if (overloadPressed || current > CURRENT_LIMIT_A ||
             (temperature != DEVICE_DISCONNECTED_C && temperature > TEMP_LIMIT_C)) {
    status = "OVERLOAD";
  } else if (normalPressed) {
    status = "NORMAL";
  }

  setDemoRelays(status == "NORMAL");

  display.clearDisplay();
  display.setCursor(0, 0);
  display.setTextSize(1);
  display.println("SMART GRID DEMO");
  display.print("Status: "); display.println(status);
  display.print("Temp: ");
  if (temperature == DEVICE_DISCONNECTED_C) display.println("Sensor error");
  else { display.print(temperature, 1); display.println(" C"); }
  display.print("Voltage: "); display.print(voltage, 2); display.println(" V*");
  display.print("Current: "); display.print(current, 2); display.println(" A*");
  display.display();

  Serial.print("{\"voltage\":"); Serial.print(voltage, 3);
  Serial.print(",\"current\":"); Serial.print(current, 3);
  Serial.print(",\"temperature\":"); Serial.print(temperature, 2);
  Serial.print(",\"status\":\""); Serial.print(status); Serial.println("\"}");

  delay(500);
}
```

ai-model/train_model.py

```
"""Train a baseline Random Forest model from labelled smart-grid examples.

Run from repository root: python ai-model/train_model.py
The bundled sample dataset is illustrative only, not for real protection.
"""
from pathlib import Path
import joblib
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.model_selection import train_test_split

ROOT = Path(__file__).resolve().parent
DATA_PATH = ROOT / "smart_grid_data.csv"
MODEL_PATH = ROOT / "smart_grid_model.joblib"
FEATURES = ["voltage", "current", "temperature"]
TARGET = "status"
REQUIRED = {"Normal", "Overload", "Fault"}

data = pd.read_csv(DATA_PATH)
missing = set(FEATURES + [TARGET]) - set(data.columns)
if missing:
    raise ValueError(f"Missing CSV columns: {sorted(missing)}")
if not REQUIRED.issubset(set(data[TARGET].unique())):
    raise ValueError(f"Dataset must include labels: {sorted(REQUIRED)}")
if data[FEATURES].isna().any().any() or data[TARGET].isna().any():
    raise ValueError("Dataset contains missing values.")

X, y = data[FEATURES], data[TARGET]
counts = y.value_counts()
if counts.min() < 2:
    raise ValueError("Need at least 2 examples per class for stratified splitting.")

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
```

```
)
model = RandomForestClassifier(
    n_estimators=150, random_state=42, class_weight="balanced"
)
model.fit(X_train, y_train)
pred = model.predict(X_test)

print("Hold-out accuracy (tiny sample data is not meaningful):",
      round(accuracy_score(y_test, pred), 3))
print(classification_report(y_test, pred, zero_division=0))
joblib.dump(model, MODEL_PATH)
print(f"Saved model to: {MODEL_PATH}")
print("Use real, diverse, correctly labelled data and independent testing before drawing
conclusions.")
```

ai-model/smart_grid_data.csv

```
voltage,current,temperature,status
6.0,0.30,28,Normal
6.1,0.40,30,Normal
5.9,0.50,32,Normal
6.0,0.35,29,Normal
6.2,0.60,34,Normal
5.8,0.45,31,Normal
5.7,2.40,48,Overload
5.6,2.70,52,Overload
5.5,2.30,55,Overload
5.4,3.00,58,Overload
5.6,2.60,50,Overload
5.3,2.80,61,Overload
0.2,0.00,29,Fault
0.3,0.05,31,Fault
0.1,0.00,30,Fault
0.4,0.02,28,Fault
0.2,0.01,33,Fault
0.0,0.00,32,Fault
```

dashboard/app.py

```
"""Streamlit demo dashboard. Values are simulated until hardware integration."""
from pathlib import Path
import streamlit as st
import pandas as pd

st.set_page_config(page_title="AI Smart Grid Digital Twin", page_icon="■", layout="wide")
st.title("■ AI-Powered Smart Grid Digital Twin")
st.caption("Monitor • Analyze • Predict • Optimize")

st.sidebar.header("Demo controls")
mode = st.sidebar.selectbox("Simulate condition", ["Normal", "Overload", "Fault"])
st.sidebar.warning("Simulation only: this page is not connected to the ESP32 yet.")

examples = {
    "Normal": {"voltage": 6.0, "current": 0.45, "temperature": 32.0},
    "Overload": {"voltage": 5.6, "current": 2.6, "temperature": 54.0},
    "Fault": {"voltage": 0.2, "current": 0.0, "temperature": 31.0},
}
reading = examples[mode]

c1, c2, c3 = st.columns(3)
c1.metric("Voltage (demo)", f'{reading["voltage"]:.2f} V')
c2.metric("Current (demo)", f'{reading["current"]:.2f} A')
c3.metric("Temperature (demo)", f'{reading["temperature"]:.1f} °C')

if mode == "Normal":
    st.success("GRID STATUS: NORMAL (SIMULATED)")
elif mode == "Overload":
    st.warning("GRID STATUS: OVERLOAD (SIMULATED)")
else:
    st.error("GRID STATUS: FAULT (SIMULATED)")

st.subheader("Sensor readings")
chart_data = pd.DataFrame({
    "Measurement": ["Voltage (V)", "Current (A)", "Temperature (°C)"],
    "Demo value": [reading["voltage"], reading["current"], reading["temperature"]],
}).set_index("Measurement")
st.bar_chart(chart_data)

st.subheader("AI prediction")
model_path = Path(__file__).resolve().parents[1] / "ai-model" / "smart_grid_model.joblib"
if model_path.exists():
    try:
        import joblib
        model = joblib.load(model_path)
        features = pd.DataFrame(
```

```
            [[reading["voltage"], reading["current"], reading["temperature"]]],
            columns=["voltage", "current", "temperature"],
        )
        prediction = model.predict(features)[0]
        st.info(f"Trained model prediction for simulated values: **{prediction}**")
        st.caption("This is a demonstration using a tiny illustrative dataset, not validated fault
detection.")
    except Exception as exc:
        st.warning(f"Could not load model: {exc}")
else:

    st.info("No trained model found yet. Run: python ai-model/train_model.py")

st.subheader("System components")
st.write("ESP32 · ZMPT101B voltage sensor · ACS712 current sensor · DS18B20 temperature sensor ·
OLED · relay module")
st.caption("All readings on this page are simulated. Real-time ESP32/Wi-Fi integration is a
separate next step.")
```

docs/PROJECT_GUIDE.md

```
# Project setup guide

## 1. Python dashboard
Install Python 3.10+ and run from the repository root:
```bash
pip install -r requirements.txt
streamlit run dashboard/app.py
```
The dashboard initially uses simulated values.

2. Train the baseline model
```bash
python ai-model/train_model.py
```
The bundled CSV is a tiny illustrative dataset for testing the workflow only. Do not use it to
claim real-world model accuracy.

3. ESP32 firmware
Open `esp32-code/SmartGrid.ino` in Arduino IDE. Install:
- Adafruit SSD1306
- Adafruit GFX Library
- OneWire
- DallasTemperature

Select the correct ESP32 board and COM/USB port. Verify OLED I2C address, relay polarity, sensor
models, and ADC input ranges before powering.

4. Hardware safety and calibration
- A 230 V AC primary is hazardous. Mains work must be done by a qualified electrician with proper
enclosure and protection.
- Only use isolated low-voltage demo loads during development.
- Never connect mains voltage to ESP32 pins.
- Check the exact ZMPT101B module input/supply specs. Its output must be safely conditioned before
the ESP32 ADC.
- ACS712 modules commonly run at 5 V; condition and verify the output before connecting to a 3.3 V
ADC.
- Calibrate voltage/current conversions against suitable references.
- Confirm the relay's default state and logic polarity with no load attached.
- Do not rely on this prototype or its AI output for real electrical protection.
```